



Memory-efficient exact bayesian network structure learning: a single-pass level-wise dynamic program

Hong-Ming Huang¹ · Joe Suzuki¹

Received: 28 July 2025 / Accepted: 26 November 2025
© The Behaviormetric Society 2025

Abstract

Bayesian network (BN) structure learning is a fundamental yet NP-hard problem in probabilistic graphical models. Dynamic programming methods, such as the algorithm proposed by Silander and Myllymaki (2012), guarantee global optimality but their practical use is limited by large memory requirements, repeated traversals of the subset lattice, and reliance on disk I/O. We propose a memory-efficient exact BN structure learning algorithm that integrates parent-set optimization and sink-node identification within a single, level-wise traversal. By retaining only the minimal information required at each level, our method eliminates disk usage and reduces peak memory complexity from $O(p2^p)$ to $O(\sqrt{p}2^p)$ while preserving $O(p^22^p)$ time complexity. Experiments show consistent improvements over the baseline dynamic program in both peak memory and runtime. Notably, without any disk support, we successfully learned a BN with 28 variables using only 16 GB of RAM. These results highlight the scalability and practical relevance of the proposed approach.

Keywords Causal discovery · Graphical models · Structure learning evaluation · Score-based methods · Global Optimality

1 Introduction

Bayesian networks (BNs) are probabilistic graphical models that represent a set of random variables and their conditional dependencies using a directed acyclic graph (DAG). In such a graph, each node corresponds to a variable, and each edge indicates a direct probabilistic dependency between variables. Owing to their strong reasoning

Communicated by Maomi Ueno.

✉ Hong-Ming Huang
hm-huang@sigmath.es.osaka-u.ac.jp

¹ Graduate School of Engineering Science, Osaka University, Osaka, Japan

capability under uncertainty, Bayesian networks have been widely applied in fields such as medicine, biology, engineering, and the social sciences. However, learning the optimal structure of a Bayesian network is an NP-hard problem (Chickering 1996). The number of candidate DAGs grows super-exponentially with the number of variables, leading to a sharp increase in computational and memory costs. These challenges make efficient and scalable structure learning a long-standing research problem of significant theoretical and practical importance.

Existing structure learning methods can be broadly categorized into two groups: constraint-based methods and score-based methods. Constraint-based methods infer network structure through conditional independence tests and are highly efficient under ideal assumptions (Spirtes and Glymour 1991; Colombo et al. 2014). However, when assumptions such as “faithfulness” or “sufficiency” are violated, or when the sample size is limited, their performance degrades significantly. Score-based methods, on the other hand, directly search the structure space for the network with the highest score, including both approximate and exact algorithms. Approximate algorithms improve scalability but can only guarantee local optima (Bouckaert 1994; Chickering 2002). In contrast, exact algorithms focus more on accuracy and reliability in structure identification. In particular, dynamic programming (DP) based algorithms (Koivisto and Sood 2004; Singh and Moore 2005; Silander and Myllymaki 2012) guarantee global optimality. However, these methods require computing and storing the parent sets for all variable subsets, resulting in exponential growth in time and memory costs, which severely limits their practical use in large-scale networks. Although previous studies have proposed various pruning and heuristic techniques to reduce the search space (De Campos et al. 2009; Yuan et al. 2011; Yuan and Malone 2013), most of them do not fundamentally optimize the DP computation process itself.

This gap motivates us to re-examine the classical DP-based exact structure learning framework and improve its core computational inefficiencies. Notably, traditional methods often rely on disk I/O to alleviate memory bottlenecks when scaling to medium and large networks, but such reliance typically introduces instability, access latency, and increased implementation complexity. To address these issues, this work builds upon (Silander and Myllymaki (2012)) and focuses on achieving efficient exact structure learning entirely in memory. Our main contributions are as follows:

- The proposed method is fully implemented in memory without relying on disk storage, effectively avoiding the instability and latency caused by disk access, thereby improving the robustness and execution efficiency of the algorithm.
- By adopting a level-by-level traversal strategy, the method retains only the necessary intermediate information at each level, significantly reducing peak memory usage and achieving more efficient resource management.
- The integration of parent set optimization and sink node identification into a single traversal process not only simplifies the learning procedure but also reduces redundant memory access, further enhancing overall computational efficiency.
- Experimental results demonstrate that the proposed method achieves substantial improvements in both time and memory efficiency. It can accurately learn Bayesian networks with up to 28 variables using only in-memory computation, with the

potential to scale further to 29 variables.

The structure of this paper is as follows: Sect. 2 introduces the background knowledge, with Sects. 2.1, 2.2, and 2.3 explaining Bayesian Network Structure Learning (BNSL), marginal likelihood, and the specific score employed, respectively. Section 3 reviews the existing work. Section 4 presents our method. Section 5 demonstrates the experimental results. Section 6 concludes with a discussion and future work directions.

2 Background

2.1 Bayesian network structure learning

Suppose that given n tuples of examples

$$X^{(1)} = x_{i,1}, X^{(2)} = x_{i,2}, \dots, X^{(p)} = x_{i,p},$$

where $i = 1, 2, \dots, n$, with respect to p variables $X^{(1)}, X^{(2)}, \dots, X^{(p)}$.

A fundamental step in structure learning is finding the optimal parent sets for each node within the network. The joint probability of the variables $X^{(1)}, \dots, X^{(p)}$ in a BN is factorized according to the network structure as follows:

$$P(X^{(1)}, \dots, X^{(p)}) = P(X_1)P(X_2|\pi_2) \cdots P(X_i|\pi_i) \cdots P(X_p|\pi_p) = \prod_{i=1}^p P(X_i | \pi_i), \quad (1)$$

where X_i denotes the vertex at i -th position in the structure and π_i denotes the parent set of the vertex X_i . We identify a BN structure that has the maximum posterior probability, given the p variables and the prior probabilities over structures and parameters. Specifically, maximizing posterior probability is achieved by considering marginal likelihood.

2.2 Marginal likelihood

In Bayesian inference, the posterior probability $P(G|D)$ represents the probability of a BN structure G given the data D . According to Bayes' theorem, the posterior probability can be expressed as

$$P(G|D) = \frac{P(D|G)P(G)}{P(D)}. \quad (2)$$

Here, $P(D|G)$ is the marginal likelihood (also known as model evidence), representing the probability of the data D given the network structure G . $P(G)$ is the prior probability, representing the prior knowledge about the network structure G before observing the data D . $P(D)$ is a normalization constant, representing the total probability of the data D .

In the structure learning process, our goal is to maximize the posterior probability $P(G|D)$ to find the most likely network structure G . However, since $P(D)$ is constant for all candidate structures G , we can ignore it when comparing different network structures. Maximizing the posterior probability is equivalent to maximizing $P(D|G)P(G)$.

$P(D|G)$ is the marginal likelihood, which considers the joint probability of all possible parameters θ . And we compute it as

$$P(D|G) := \int_{\Theta} P(D|G, \theta) P(\theta|G) d\theta. \quad (3)$$

This means that we not only consider the likelihood of the data given the network structure $P(D|G, \theta)$, but also integrate the uncertainty of the parameters and the prior distribution $P(\theta|G)$. Therefore, the process of maximizing the posterior probability $P(G|D)$ is actually achieved by maximizing the marginal likelihood $P(D|G)$ while also considering the prior probability $P(G)$.

We then use $Q(X)$ to represent the marginal likelihood $P(D|G)$. More specifically, for a set of realized values $\mathcal{X}$ for a random variable X , and a parameter set $\Theta \subseteq \mathbb{R}^d$. Given a sample size n and i.i.d. observations $x_1, \dots, x_n \in \mathcal{X}$, the model is described by a family of probability distributions $\{p(\cdot|\theta)\}_{\theta \in \Theta}$ and a prior distribution φ over Θ . The marginal likelihood is defined as

$$Q(X) := \int_{\Theta} \varphi(\theta) \prod_{i=1}^n p(x_i|\theta) d\theta, \quad (4)$$

which synthesizes the likelihood across all parameter values, weighted by the prior. According to Eq. (1), let us set $V = \{X^{(1)}, \dots, X^{(p)}\}$. The maximum posterior probability for identifying a BN structure can then be expressed as

$$R(V) := \max_{\pi_i} \prod_{i=1}^p \frac{Q(X_i, \pi_i)}{Q(\pi_i)}, \pi_i \subseteq V \setminus \{X_p, \dots, X_i\}. \quad (5)$$

2.3 Quotient Jeffreys' score

In this paper, we focus on multivariate and complete discrete data, and each variable has a finite number of values. For score-based approaches, the specific score is selectable. Before providing a detailed introduction, we first clarify several definitions. A score is considered normal if it ensures that all Markov equivalent Bayesian network structures receive the same score. Regularity refers to the property that the local score of a model should not increase when additional conditioning variables are added, provided that these variables already uniquely determine the target variable. This definition reflects the reasonable penalization of model complexity by the scores. Commonly used scores are divided into two categories: information-theoretic scores and Bayesian scores.

Information-theoretic scores do not require subjective prior parameter specification, making them simple to use and computationally efficient. Common examples include AIC (Akaike 1974), BIC (Suzuki 1993; SUZUKI 1999), and MIT (de Campos 2006). Among these, BIC, also known as MDL, is particularly common and is supported by a strong theoretical foundation. It provides effective pruning for redundant parent sets, and many algorithms have been developed for efficient Bayesian network structure learning based on its properties (Malone et al. 2011; Yuan et al. 2011; Yuan and Malone 2013). Bayesian scores, on the other hand, incorporate prior beliefs on both the network structure and its parameters, quantifying the posterior probability of different network structures given the data. Typical examples include BD, BDe, and BDeu, with the latter two being normal (Buntine 1991; Heckerman et al. 1995). Among these, BDeu is widely used but does not satisfy regularity (Suzuki and Kawahara 2017). In addition, there are two types of scores based on Jeffreys' prior: the first directly uses Jeffreys' prior (Jeffreys 1998; Krichevsky and Trofimov 1981), while the second is the quotient score based on Jeffreys' prior (Suzuki and Kawahara 2017). The former satisfies regularity but not normality, while the latter satisfies both properties. Since this study focuses on optimizing the basic framework of existing methods, any score is applicable. As pruning is not involved in this context, we ultimately select the quotient score based on Jeffreys' prior as our score here.

According to the knowledge about marginal likelihood and Jeffreys' prior, the marginal likelihood $Q(X)$ shown above can be simplified as

$$Q(X) := \prod_{i=1}^n \frac{c_{i-1}(x_i) + 0.5}{i - 1 + 0.5\sigma(X)}, \quad (6)$$

where $c_{i-1}(x)$ is the number of occurrences of x in $(x_1, \dots, x_{i-1})$ which starts from 0, and $\sigma(X)$ equals the number of different values that X takes. When computing the parent set, we apply quotient Jeffreys' score as

$$Q(X|Y) = \frac{Q(X, Y)}{Q(Y)}. \quad (7)$$

For example, if $(x_1, \dots, x_5) = (0, 1, 0, 1, 1)$ and $(y_1, \dots, y_5) = (0, 0, 1, 1, 1)$, then we have

$$\begin{aligned} Q(X) &= \frac{0+0.5}{0+0.5 \cdot 2} \cdot \frac{0+0.5}{1+0.5 \cdot 2} \cdot \frac{1+0.5}{2+0.5 \cdot 2} \cdot \frac{1+0.5}{3+0.5 \cdot 2} \cdot \frac{2+0.5}{4+0.5 \cdot 2}, \\ Q(Y) &= \frac{0+0.5}{0+0.5 \cdot 2} \cdot \frac{1+0.5}{1+0.5 \cdot 2} \cdot \frac{0+0.5}{2+0.5 \cdot 2} \cdot \frac{1+0.5}{3+0.5 \cdot 2} \cdot \frac{2+0.5}{4+0.5 \cdot 2}, \\ Q(X, Y) &= \frac{0+0.5}{0+0.5 \cdot 4} \cdot \frac{0+0.5}{1+0.5 \cdot 4} \cdot \frac{0+0.5}{2+0.5 \cdot 4} \cdot \frac{0+0.5}{3+0.5 \cdot 4} \cdot \frac{1+0.5}{4+0.5 \cdot 4}, \\ \frac{3}{256} &= Q(X) > \frac{Q(X, Y)}{Q(Y)} = Q(X | Y) = \frac{1}{90}. \end{aligned}$$

Here, when we consider the optimal parent set for each variable in the set $\{X, Y\}$, Y is not included in the parent set of X .

3 Existing work

The state-of-the-art algorithm progresses through several steps to achieve the final globally optimal solution (Silander and Myllymaki 2012), as shown in Fig. 1.

The algorithm begins by calculating every possible local score $Q(S)$ for all subsets $S \subseteq \{X^{(1)}, \dots, X^{(p)}\}$. Secondly, for each subset S and $X \in S$, the optimal parent set for each node in it is computed by

$$\begin{aligned} \frac{Q(X, \pi(X, S \setminus X))}{Q(\pi(X, S \setminus X))} &= \max_{T \subseteq S \setminus X} \frac{Q(X, T)}{Q(T)} \\ &= \max \left\{ \frac{Q(X, S \setminus X)}{Q(S \setminus X)}, \max_{Y \in S \setminus X} \frac{Q(X, \pi(X, S \setminus \{X, Y\}))}{Q(\pi(X, S \setminus \{X, Y\}))} \right\}, \end{aligned} \quad (8)$$

where $\pi(X, S \setminus X)$ is the parent set of $X \in S$ within $S \setminus X$. When performing the computation entirely in memory, the two steps described above require storing $2^p - 1$ local scores and $p2^{p-1}$ parent set entries, respectively. The second term $p2^{p-1}$ arises because for p variables, each variable must compute scores for all possible parent set combinations to ensure that the global search can later retrieve the optimal solution directly. However, this process consumes a substantial amount of memory. To address this issue, this study introduces a level-based computation strategy. We partition the subsets corresponding to different combination sizes into levels, ensuring that when computing level k (i.e., all subsets containing k variables), the algorithm only depends on the information from level $k - 1$. In other words, the optimal parent sets and their corresponding scores for each level can be incrementally derived from the results of the previous level, without the need to store all data in memory simultaneously.

Thirdly, the results from the previous calculations are used for finding the sink nodes. By identifying the sink node for each local structure step by step and then combining the sink nodes corresponding to all variables, the optimal order for global structure can be inferred. For example, for the set $\{X, S\}$, if X is known to be the sink variable, and its optimal parent set has been determined in the second step, the information for X is confirmed for this local structure. Next, only S needs to be considered.

Fourthly, by utilizing the identified sink nodes to deduce the order. For the set $\{X, Y, Z\}$, if Z is the sink node and Y is the sink node for $\{Y, X\}$, then X is the most upstream node and has no parent set. The parent sets of Y and Z can be directly



Fig. 1 Five essential steps in the state-of-the-art algorithm

obtained from the corresponding results saved in the second step. Step 3 and Step 4 can be summarized by the following equation

$$R(S) := \begin{cases} \max_{X \in S} R(S \setminus X) \frac{Q(X, \pi(X, S \setminus X))}{Q(\pi(X, S \setminus X))}, & |S| \geq 1 \\ 1, & |S| = 0 \end{cases} \quad (9)$$

Finally, based on the obtained order, by considering the optimal parent set within the corresponding local structure, the optimal Bayesian network can be output.

In the entire process, it is necessary to traverse all combinations three times: the first for computing local scores, the second for determining the optimal parent sets, and the third for identifying the optimal sink nodes. Although the first and second steps can be combined by calculating the required local scores while determining the optimal parent sets and directly reusing them if they have already been computed, this still requires traversing all combinations at least twice. Although the theoretical time complexity remains the same, the difference in access patterns and memory usage can have a significant impact in practice. If the entire process is fused into a single traversal of all subsets, it can reduce memory access overhead and minimize intermediate data read and write operations, thereby significantly improving the actual runtime efficiency. Through the level-based computation strategy mentioned above, we will integrate the three processes into a single procedure. In the next section, we will specifically discuss why this optimization can contribute to improving computational efficiency.

4 Proposed method

The core concept of the proposed method is illustrated in Fig. 2.

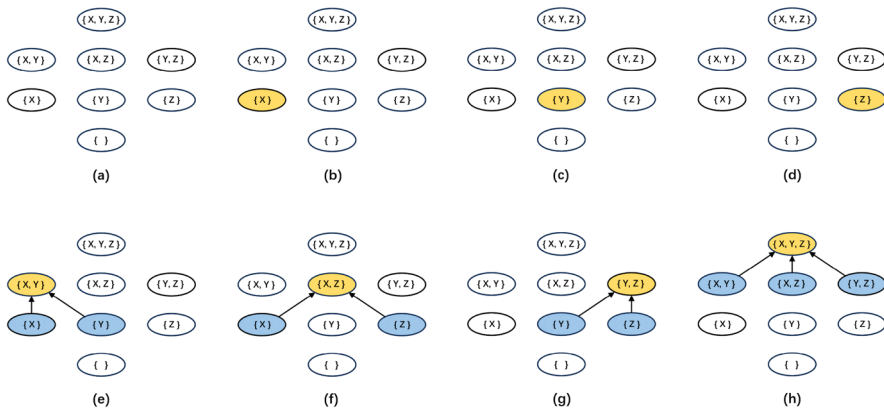


Fig. 2 Illustration of the level-by-level computation process in the proposed algorithm. Yellow indicates the combination being computed, while blue represents the data and information required for the computation

Figure 2a displays all subsets of the set $\{X, Y, Z\}$. The computation process proceeds level by level as follows:

- First Level (Fig. 2b–d): No information is needed from the empty set. Each subset's data is computed individually.
- Second Level (Fig. 2e–g): In Fig. 2e, to compute the set $\{X, Y\}$, data from both $\{X\}$ and $\{Y\}$ is required. However, for Fig. 2f, the data from $\{X\}$ is also needed. Thus, the data saved from the first level is utilized, avoiding redundant computations. During the computation process, the data from the first level is updated and saved into the second level.
- Third Level (Fig. 2h): This level involves the complete set of variables. At this stage, only the data from the second level is needed to compute the final results, without referring back to the first level.

Next, we will provide the detailed computation process for each set. For every subset S where $|S| = k$, we assume that $R(S)$ and $\pi(X, S \setminus X)$ for each X in S have already been computed in the former level, the process is then repeated for each subset S where $|S| = k + 1$. The following is the description of the algorithm steps:

1. We firstly compute $Q(S)$ for this local structure.
2. For each $X \in S$, we compute $Q(S \setminus X)$. And we define $\pi(X, S \setminus X)$ by

$$\pi(X, S \setminus X) := \max \left\{ \frac{Q(S)}{Q(S \setminus X)}, \max_{Y \in S \setminus X} \frac{Q(X, \pi(X, S \setminus \{X, Y\}))}{Q(\pi(X, S \setminus \{X, Y\}))} \right\}. \quad (10)$$

Here, when $\frac{Q(S)}{Q(S \setminus X)}$ is maximized, $\pi(X, S \setminus X)$ retains the corresponding value and sets the optimal parent set of X in this structure as $S \setminus X$. When there exists a $Y \in S \setminus X$ such that $\max_{Y \in S \setminus X} \frac{Q(X, \pi(X, S \setminus \{X, Y\}))}{Q(\pi(X, S \setminus \{X, Y\}))}$ is the maximum value, $\pi(X, S \setminus X)$ retains the corresponding value and updates the optimal parent set of X in this structure with the information from $\pi(X, S \setminus \{X, Y\})$.

3. Finally, we will find the $X \in S$ that maximizes $R(S \setminus X) \frac{Q(X, \pi(X, S \setminus X))}{Q(\pi(X, S \setminus X))}$ to obtain $R(S)$. That is, to find the sink node of S .

In this manner, the optimal structure is consistently obtained for each subset S , with the optimal parent set for each variable continuously updated, ensuring that we ultimately achieve a globally optimal result. The proposed algorithm is summarized in Algorithm 1. To keep the pseudocode concise and easy to read, we have omitted two implementation details. Specifically, the operations in Lines 19 and 24 not only correspond to updating the numerical value of the optimal parent set for the current variable X (Line 19) and updating the numerical value of the optimal structure for the current subset S (Line 24), but also involve recording structural information using masks (i.e., bitwise operations). More concretely, when executing $\pi(X, S') \leftarrow \pi(X, S' \setminus \{Y\})$, we record that for variable X , the optimal parent set in the subset S' corresponds to $S' \setminus \{Y\}$. Similarly, when executing $R(S) \leftarrow \text{cand}$, we

record that variable X is the sink node within the current subset S . All of this additional information is stored in $\mathcal{M}_\ell$ and continuously updated as the level changes. In the final stage, the Bayesian network structure can be precisely reconstructed based on the parent set and sink node information recorded in the masks.

Require: Variable set $\mathcal{V} = \{X_1, \dots, X_p\}$

Ensure: Optimal global score $R(\mathcal{V})$ and parent sets $\pi(X, \mathcal{V} \setminus \{X\})$ for each $X \in \mathcal{V}$

```

1: Compute  $R(X_1) = Q(X_1), \dots, R(X_p) = Q(X_p)$ 
2: Store the results in DP storage  $\mathcal{M}_1$ 
3: Initialize level  $\ell \leftarrow 2$ 
4: while  $\ell \leq p$  do                                     ▷ Bottom-up DP over subset size
5:   for all subsets  $S \subseteq \mathcal{V}$  with  $|S| = \ell$  do
6:      $Q(S) \leftarrow$  Local score of subset  $S$ 
7:      $R(S) \leftarrow -\infty$ 
8:     for all  $X \in S$  do                                     ▷ Enumerate sink candidate
9:        $S' \leftarrow S \setminus \{X\}$ 
10:      Obtain  $Q(S'), R(S')$  from  $\mathcal{M}_{\ell-1}$ 
11:       $\pi(X, S') \leftarrow Q(S)/Q(S')$ 
12:      for all  $Y \in S'$  do                                     ▷ Update the optimal parent set
13:        if  $\pi(X, S' \setminus \{Y\})$  exists in  $\mathcal{M}_{\ell-1}$  then
14:          Obtain  $\pi(X, S' \setminus \{Y\})$  from  $\mathcal{M}_{\ell-1}$ 
15:        else
16:           $\pi(X, S' \setminus \{Y\}) \leftarrow Q(X)$ 
17:        end if
18:        if  $\pi(X, S' \setminus \{Y\}) > \pi(X, S')$  then
19:           $\pi(X, S') \leftarrow \pi(X, S' \setminus \{Y\})$ 
20:        end if
21:      end for
22:       $\text{cand} \leftarrow R(S') \times \pi(X, S')$ 
23:      if  $\text{cand} > R(S)$  then                                     ▷ Update the optimal state of  $S$ 
24:         $R(S) \leftarrow \text{cand}$ 
25:      end if
26:    end for
27:    Store  $\{Q(S), R(S), \{\pi(X, S \setminus X) : X \in S\}\}$  into  $\mathcal{M}_\ell$ 
28:  end for
29:  Delete  $\mathcal{M}_{\ell-1}$ 
30:   $\ell \leftarrow \ell + 1$ 
31: end while
32: return  $R(\mathcal{V})$  and  $\{\pi(X, \mathcal{V} \setminus X) : X \in \mathcal{V}\}$ 

```

Algorithm 1 Level-by-level dynamic programming for optimal BN structure

For existing work, they claim that the memory complexity of the algorithm is $O(2^p)$, which is accurate when using a hard disk to store the best parent sets. However, when using only memory, the complexity becomes $O(p2^p)$. In this scenario, a computer with 16GB of memory can handle a Bayesian network with a maximum of 26 variables. Through theoretical analysis, we have obtained the following evaluation results.

Although the improvement is only by a factor of $\frac{1}{\sqrt{p}}$, a process that originally required 64GB of memory can now be accomplished with 16GB of memory, highlighting its value. The detailed proof for Table 1 can be found in A.

In theory, the time complexity of the two methods is identical. However, as we pointed out in Sect. 3, our proposed algorithm consolidates the multiple traversals of all subsets in previous methods into a single pass, thereby avoiding unnecessary

Table 1 Comparison of computation and memory requirements

	Silander–Myllymäki	Proposed method
Computation	$O(p^2 2^p)$	$O(p^2 2^p)$
memory	$O(p 2^p)$	$O(\sqrt{p} 2^p)$

memory accesses and improving computational efficiency. This improvement is also validated by the experimental results presented in Sect. 5. In the following, we further illustrate this with a concrete example.

Example 1 (*Three-variable case: $\{X, Y, Z\}$*) Consider three variables $\{X, Y, Z\}$. In existing methods, when computing the optimal parent set for each subset S , the corresponding $\pi(X, S')$ is first calculated and stored for later use. During the subsequent sink node identification stage, all subsets must be traversed again to retrieve this stored information.

For example, in the case of the two-variable subsets $\{X, Y\}$, $\{X, Z\}$, and $\{Y, Z\}$, after obtaining the optimal parent sets $\pi(X, Y)$, $\pi(Y, X)$, and so on, the existing method simply stores them without immediately updating the corresponding local optimal solution $R(\cdot)$. As a result, during the sink node identification stage, it needs to read the stored optimal parent sets an additional $2 \times 3 = 6$ times. For the three-variable subset $\{X, Y, Z\}$, it similarly needs to read the optimal parent sets $\pi(X, \{Y, Z\})$, $\pi(Y, \{X, Z\})$, and $\pi(Z, \{X, Y\})$ three more times to complete the sink node update.

In contrast, the proposed method updates the local optimal solution $R(S)$ immediately whenever the optimal parent set is updated, instead of separating the two steps. For both two-variable and three-variable subsets, once $\pi(\cdot)$ is obtained, it is directly combined with the existing $R(\cdot)$ to update the local optimal solution of the current subset, without waiting for a second traversal.

This synchronous update strategy allows the entire process to traverse all subsets only once to complete both parent set selection and local solution updates. It avoids repeated accesses to parent sets and local solutions during the sink node identification stage, thereby reducing memory access and improving the actual runtime efficiency.

From a theoretical perspective, at each level, the number of avoided accesses to the local optimal parent sets in the proposed method is equal to the product of the number of elements in the current set and the number of subsets at that level. Since at the first level we do not need to compute any optimal parent sets, the total number of avoided accesses can be expressed as

$$\sum_{i=2}^p i \times \binom{p}{i}.$$

5 Experiments

Due to the possibility of exceeding 16GB during the experiments, we used a Core i7-12700K CPU with a 32GB DDR4 memory kit from G.Skill for the relevant experiments in this paper. The algorithm is executed via Rcpp in the R language (Eddelbuettel and François 2011), which can effectively enhance computational efficiency. Considering our exploration of the upper limit of the number of variables, we used the Alarm database with 37 variables (Beinlich et al. 1989). The sample size will not affect the comparison results between the two algorithms, and it is not a limiting factor for Bayesian networks. In the specific experiments, since we mainly compare the time and memory complexity of the two algorithms, we will simply set the sample size to 200.

In Sect. 5.1, we will present the experimental results comparing the two methods to demonstrate that the proposed method is more advantageous in terms of time and memory complexity. In Sect. 5.2, we will show the stability of the proposed method when using memory only. In Sect. 5.3, we will demonstrate the implementation of a Bayesian network with 28 variables using the proposed method with memory only.

5.1 Comparison of two methods

Based on Table 1 and our calculations, we can determine the maximum number of variables that can be handled by the two methods when using only 16GB of memory. For the existing method, the upper limit is 26 variables, whereas our proposed method can handle up to 28 variables. As mentioned in Sect. 1, another paper proposed an improvement to existing methods (Malone et al. 2011). However, we have also pointed out that their framework still separates the computation of optimal parent sets from node ordering, and stores the computed optimal parent sets on disk. Without the use of disk storage, their method may perform slightly better than that of Silander and Myllymaki (2012), but it is clearly outperformed by our proposed approach. Therefore, we do not include additional experiments on their method here. To ensure consistency, since both our proposed method and the existing method can implement Bayesian networks with up to 25 variables, our comparative experiments will start from 20 variables and go up to 25 variables. For fewer than 20 variables, the required memory and computation time are negligible. For each number of variables, each algorithm will be run ten times, and the average result will be reported.

As shown in Fig. 3a, the peak memory usage of the proposed method is significantly lower than that of the existing method across different numbers of variables. This validates the superiority of the proposed method in terms of memory usage. As shown in Fig. 3b, even though the theoretical computational complexity is similar, the algorithm is more efficient, resulting in a noticeable improvement in runtime.

The specific data corresponding to different numbers of variables is detailed in Table 2. The table illustrates how the proposed method consistently outperforms the existing method in both memory usage and execution time. This performance advantage becomes increasingly significant as the number of variables increases, highlighting the scalability and robustness of the proposed approach.

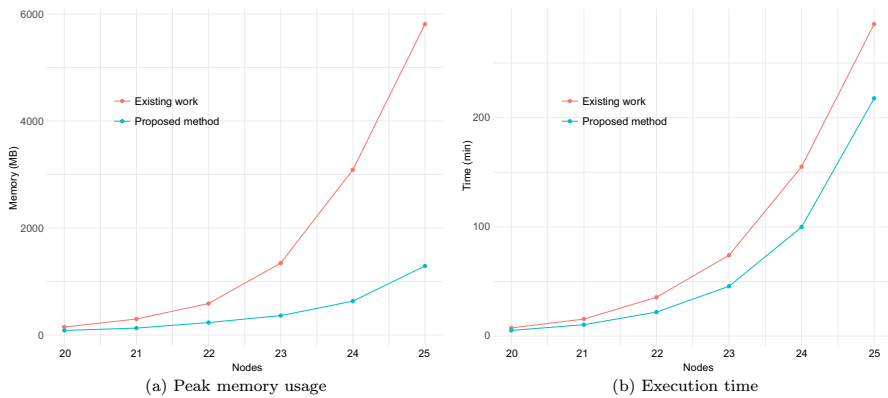


Fig. 3 Comparison of different metrics

Table 2 Average data for variables within the range of 20 to 25 in detail

Number of variables	Time (min)		Memory (MB)	
	Existing work	Proposed method	Existing work	Proposed method
20	7.50	5.21	148.43	84.86
21	15.63	10.46	298.33	128.30
22	35.55	21.96	588.16	232.00
23	74.03	45.69	1343.69	362.12
24	155.10	99.85	3088.90	632.11
25	285.67	217.70	5809.79	1289.59

5.2 Stability test of the proposed method

In this subsection, we will present the experimental results of the proposed method, verifying its stability in terms of time and memory complexity when using only memory. We will present the comparison between the results of ten experiments and their average values. The specific data will be presented in Tables 3 and 4 in Appendix B. For memory complexity, by properly allocating memory resources and effectively managing memory usage, unnecessary memory leaks and overflows can be avoided, thereby enhancing the stability and performance of the program. As shown in Fig. 4a, although there are some fluctuations during code execution, the memory usage remains relatively stable. For runtime, as shown in Fig. 4b, the runtime remains stable regardless of the number of variables.

5.3 Constructing a large network

Theoretically, the maximum Bayesian network that can be run using only memory is 28 variables, so we chose to implement this. As mentioned in the experimental details of this section, we will use the first 28 variables of the Alarm dataset and a sample size of 200 for computation. The final output results are shown in Fig. 5.

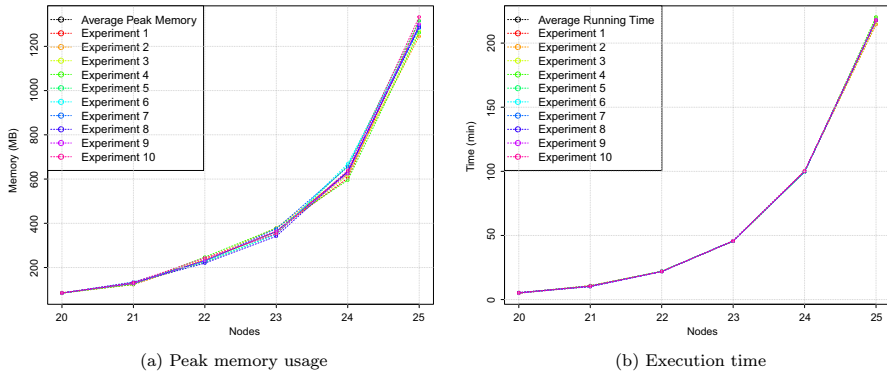


Fig. 4 Verification of the stability of the proposed method

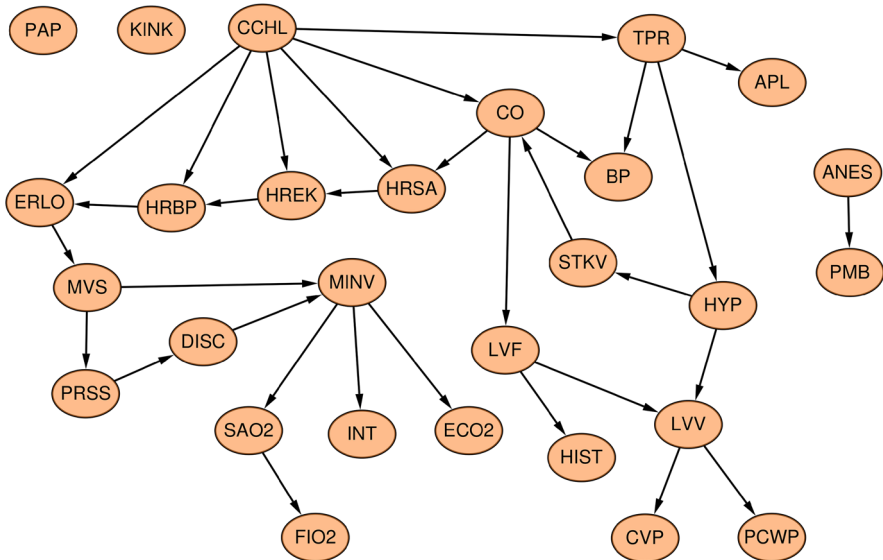


Fig. 5 The output of running on the first 28 variables of the Alarm dataset

We used the hardware equipment mentioned at the beginning of this section to run this code. The specific experiment took 1944.43 min, and the peak memory usage was 10070.62MB, which is obvious that it is impossible to construct a Bayesian network with 29 variables using only 16GB memory. However, for the proposed method, it seems that there are still some interesting points worth analyzing.

As shown in Fig. 6, when we solve for the optimal Bayesian network level by level, we are still constrained by the number of combinations. It can be seen that the number of combinations near the middle level is much larger than at both ends. For the case of 29 variables, it can be inferred that the peak memory consumption occurs at the 15th level. Specifically, the most memory-consuming part is storing the optimal parent set vector for each variable. Although it is the 15th level, the parent set of a

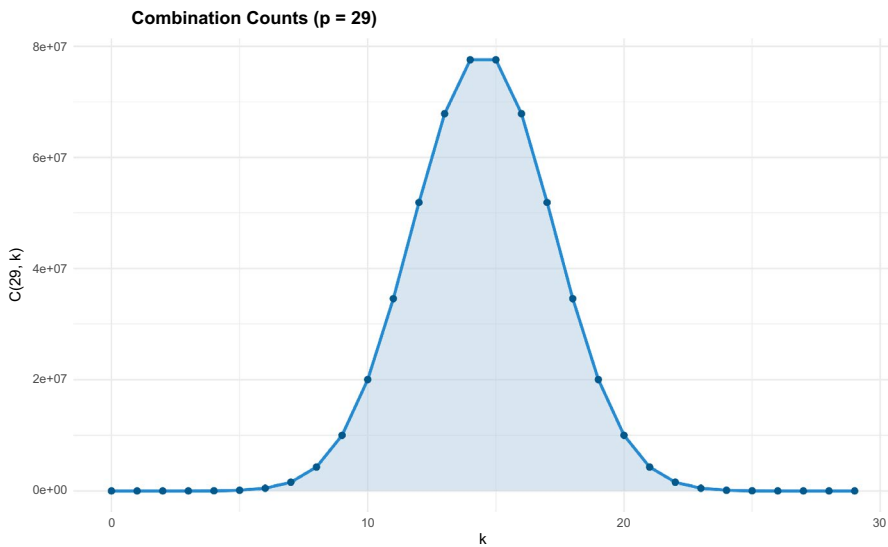


Fig. 6 The change in the number of combinations when the number of variables is 29

variable cannot include itself, so we first need to calculate the combination number $C(28, 14)$, then multiply by the number of variables. In this case, storing both the current optimal parent set data and related information would require more than 25 GB of memory. However, if only the essential information is retained, memory usage can be reduced to less than 10 GB, making it entirely feasible to learn a Bayesian network with 29 variables in memory. The main challenge with this approach, however, lies in the fact that it leads to a significant increase in computational overhead, especially at the highest level and the two adjacent levels, due to the substantial amount of additional computations required. Therefore, although this is a feasible option, it necessitates a trade-off between memory consumption and computational resources in practical applications.

From another perspective, if the optimal parent set vectors of a particular level are stored on disk, constructing a Bayesian network with 29 variables becomes considerably easier. Compared with existing approaches, the proposed method only requires the use of disk storage at or near the peak memory levels, rather than throughout the entire process, which helps to reduce peak memory consumption. For instance, if disk storage is utilized at the 15th level, the remaining data and information for the current level occupy less than 2 GB. The usage of disk storage can also be further optimized, such as by storing only the optimal parent sets for a subset of variables on disk, thereby minimizing interactions with the disk as much as possible.

Moreover, the proposed method can be flexibly combined with different scores to efficiently manage the optimal parent sets, and this theoretical upper bound can be directly employed for pruning in practical algorithm design. Specifically, when the number of candidate parents for a node exceeds this bound, further expansion is unnecessary, which effectively reduces the memory footprint required for storing parent sets. Since this study focuses primarily on improving the foundational framework

relative to existing methods, we will further explore the performance of our method under different scores and pruning strategies, as well as its potential for application to larger-scale datasets and more complex network structures, in future work.

6 Conclusion

Bayesian networks Structure learning is a fundamental problem in the field of probabilistic graphical models and has been widely applied across numerous scientific domains. However, the NP-hardness of this problem and the combinatorial explosion caused by increasing variable numbers have posed persistent challenges in terms of computation and memory requirements. While traditional approaches have made certain progress, they either rely heavily on disk storage—resulting in instability and latency—or fail to guarantee global optimality. Therefore, developing more efficient, scalable, and memory-friendly exact algorithms holds significant theoretical and practical value for the advancement of the field.

In this study, we propose a globally optimal Bayesian network structure learning algorithm based on a progressively leveled scoring approach, which integrates parent set optimization and sink node identification into a single-pass process. By retaining only the minimal information required at each step, our method eliminates dependence on disk storage, enabling efficient memory management entirely in main memory while ensuring algorithmic exactness. Through extensive large-scale experiments, we systematically evaluate the scalability and efficiency of the proposed method and compare it with existing approaches.

The main contributions of this study are as follows: First, by integrating parent set optimization with sink node identification, we substantially simplify the structure learning process. Second, our method achieves exact structure learning for Bayesian networks with up to 28 variables using only 16GB of memory, surpassing previous memory-efficient approaches. Third, the proposed method not only reduces peak memory usage but also allows for the flexible integration of disk support at critical stages, enabling the construction of 29-variable networks and further mitigating the shortcomings of conventional disk-based methods. In addition, the upper bound on candidate parent sets at each level provides a practical basis for subsequent algorithmic pruning.

Despite these advances, our method has certain limitations. As the variable scale continues to grow, the algorithm incurs considerable computational overhead in intermediate stages, leading to a practical trade-off between memory and computational resources. Moreover, although the method can leverage disk storage at key stages, disk operations may still impact overall efficiency in extremely large-scale applications. Future research will focus on further optimizing the balance between memory and computational complexity, extending support for more scoring functions and pruning strategies, and validating and enhancing the robustness and scalability of the algorithm on more complex and larger real-world datasets.

Appendix: Proof of time and space complexity

Assuming there are a total of p variables, for the k -th level, where the sets contain k variables. We here consider time complexity firstly.

1. The first step computes the local score with a time complexity of $\binom{p}{k}$.
2. In the second step, for each set, we need to update the optimal parent set for each variable. According to Equation (10), the time complexity is

$$\binom{p}{k} \binom{k}{1} \binom{k-1}{1}.$$

3. In the third step, using the data from the current and previous layers, we compute the sink node for each set. The time complexity is

$$\binom{p}{k} \binom{k}{1}.$$

We aim to compute the complexity of the following sum:

$$\sum_{k=1}^p \left[\binom{p}{k} + \binom{p}{k} k(k-1) + \binom{p}{k} k \right].$$

We start by breaking this sum into three parts. First, considering the binomial theorem, we simply get

$$\sum_{k=1}^p \binom{p}{k} = 2^p - 1.$$

Next, we consider the sum:

$$\sum_{k=1}^p \binom{p}{k} k(k-1).$$

Using the identity $k(k-1) \binom{p}{k} = p(p-1) \binom{p-2}{k-2}$, the sum becomes

$$\sum_{k=1}^p k(k-1) \binom{p}{k} = \sum_{k=2}^p k(k-1) \binom{p}{k} = p(p-1) \sum_{k=2}^p \binom{p-2}{k-2}.$$

Changing the index of summation:

$$\sum_{k=2}^p \binom{p-2}{k-2} = \sum_{j=0}^{p-2} \binom{p-2}{j} = 2^{p-2}.$$

Thus, we have

$$\sum_{k=1}^p k(k-1) \binom{p}{k} = p(p-1) \cdot 2^{p-2}.$$

Similar to the above derivation process, we can obtain the third part as follows:

$$\sum_{k=1}^p k \binom{p}{k} = p \sum_{k=1}^p \binom{p-1}{k-1} = p \sum_{j=0}^{p-1} \binom{p-1}{j} = p \cdot 2^{p-1}.$$

Combining all three parts, we obtain

$$\sum_{k=1}^p \left[\binom{p}{k} + \binom{p}{k} k(k-1) + \binom{p}{k} k \right] = (2^p - 1) + p(p-1) \cdot 2^{p-2} + p \cdot 2^{p-1}.$$

Therefore, the time complexity is $O(p^2 2^p)$.

Next, we calculate the memory complexity. For binomial coefficients, the middle value $\binom{p}{k}$ reaches its maximum when k is approximately $\frac{p}{2}$. However, we must consider the vector that stores the optimal parent set, which corresponds to the scenario where $k \frac{p}{k}$ reaches its maximum value. Therefore, we need further analysis.

Stirling's approximation provides an approximate value for $n!$:

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n. \quad (11)$$

Using Stirling's approximation, we obtain

$$k \binom{p}{k} = \frac{k \cdot p!}{k!(p-k)!} \approx \frac{k \sqrt{2\pi p} \left(\frac{p}{e}\right)^p}{\sqrt{2\pi k} \left(\frac{k}{e}\right)^k \cdot \sqrt{2\pi(p-k)} \left(\frac{p-k}{e}\right)^{p-k}}.$$

After simplification, let $f(k) = \frac{k \sqrt{2\pi p}}{2\pi \sqrt{k(p-k)}} \cdot \frac{p^p}{k^k (p-k)^{p-k}}$. Taking the natural logarithm, we get:

$$\ln f(k) = \ln \left(\frac{k\sqrt{2\pi p}}{2\pi\sqrt{k(p-k)}} \right) + \ln \left(\frac{p^p}{k^k(p-k)^{p-k}} \right).$$

Then we can Simplify it as

$$\ln f(k) = \ln k + \frac{1}{2} \ln(2\pi p) - \ln(2\pi) - \frac{1}{2} \ln k - \frac{1}{2} \ln(p-k) + p \ln p - k \ln k - (p-k) \ln(p-k).$$

Taking the derivative with respect to k :

$$\begin{aligned} \frac{d}{dk} \ln f(k) &= \frac{1}{k} - \frac{1}{2k} + \frac{1}{2(p-k)} - \ln k - 1 + \ln(p-k) + 1 \\ &= \frac{1}{2k} + \frac{1}{2(p-k)} - \ln k + \ln(p-k) \end{aligned}$$

At this point, we need to find the value of k such that the following equation holds

$$\frac{1}{2k} + \frac{1}{2(p-k)} - \ln k + \ln(p-k) = 0.$$

In fact, it is quite clear that the point at which the maximum value is achieved is slightly greater than $k = \frac{p}{2}$. However, in our actual situation, we take the value as an integer. Therefore, to obtain the memory complexity, we can simply take $k = p/2$ for the calculation.

Using Stirling's approximation, we obtain

$$\frac{p}{2} \left(\frac{p}{2} \right) \approx \frac{p}{2} \cdot \frac{\sqrt{2\pi p} \left(\frac{p}{e} \right)^p}{\left(\sqrt{\pi p} \left(\frac{p}{2e} \right)^{\frac{p}{2}} \right)^2} = \frac{2^p \sqrt{2\pi p}}{2\pi}.$$

It proves that the algorithm's memory complexity is $O(\sqrt{p}2^p)$.

Appendix B: Detailed experimental data results of the proposed method

See Table 3 and 4

Table 3 Average runtime and experimental results (in minutes)

Variables	Avg Time	Exp 1	Exp 2	Exp 3	Exp 4	Exp 5
20	5.206	4.946	5.021	5.278	5.344	5.013
21	10.461	10.877	10.159	10.905	10.473	10.535
22	21.965	21.899	22.051	22.083	22.110	21.958
23	45.692	45.587	45.642	45.629	45.620	45.609
24	99.847	100.245	99.679	99.925	99.506	99.692
25	217.699	214.567	219.227	215.028	220.421	218.180
Variables		Exp 6	Exp 7	Exp 8	Exp 9	Exp 10
20		5.269	5.312	5.492	5.005	5.382
21		10.127	10.675	10.051	10.292	10.514
22		22.058	21.826	21.863	21.807	21.992
23		45.928	45.623	45.814	45.643	45.830
24		99.539	99.791	99.767	100.079	100.252
25		218.271	217.840	217.612	218.195	217.649

Table 4 Average peak memory usage and experimental results (in MB)

Variables	Avg Peak Memory	Exp 1	Exp 2	Exp 3	Exp 4	Exp 5
20	84.857	84.814	86.135	83.804	85.110	84.583
21	128.304	123.250	128.234	124.374	123.120	128.951
22	231.996	230.666	236.945	228.125	247.300	223.971
23	362.122	365.414	378.747	363.225	378.199	351.255
24	632.109	603.989	625.571	616.573	595.036	659.739
25	1289.586	1315.434	1245.000	1268.855	1261.305	1314.414
Variables		Exp 6	Exp 7	Exp 8	Exp 9	Exp 10
20		85.185	84.274	84.645	85.058	84.960
21		129.622	134.812	131.821	127.125	131.727
22		228.703	229.264	219.751	231.889	243.350
23		350.183	375.353	341.457	364.786	352.606
24		669.205	654.691	634.061	623.596	638.635
25		1281.074	1292.387	1284.350	1299.879	1333.159

Author Contributions HMH: methodology, theory, coding, and writing; JS: methodology, theory validation, and supervision. All authors discussed the theoretical and experimental results and contributed to the final manuscript.

Funding HMH was supported by JST SPRING, Grant Number JPMJSP2138.

JS was supported by the Grants-in-Aid for Scientific Research (C) under Grant Number 22K11931.

Data Availability Not applicable.

Declarations

Conflict of interest The authors declare no Conflict of interest.

Ethics approval Not applicable.

Consent to participate Not applicable.

Consent for publication Not applicable.

References

- Akaike H (1974) A new look at the statistical model identification. *IEEE Trans Autom Control* 19(6):716–723. <https://doi.org/10.1109/TAC.1974.1100705>
- Beinlich IA, Suermondt HJ, Chavez RM, Cooper GF (1989) The alarm monitoring system: a case study with two probabilistic inference techniques for belief networks. In: *AIME 89: Second European Conference on Artificial Intelligence in Medicine*, London, August 29th–31st 1989. Proceedings, Springer, pp. 247–256
- Bouckaert RR (1994) Properties of bayesian belief network learning algorithms. In: *Uncertainty in Artificial Intelligence*, Elsevier, pp 102–109
- Buntine W (1991) Theory refinement on bayesian networks. In: *Uncertainty proceedings 1991*, Elsevier, pp. 52–60
- de Campos LM (2006) A scoring function for learning bayesian networks based on mutual information and conditional independence tests. *J Mach Learn Res* 7:2149–2187
- Chickering DM (1996) Learning bayesian networks is np-complete. In: *Artificial intelligence and statistics V, Learning from data*, pp. 121–130
- Chickering DM (2002) Optimal structure identification with greedy search. *J Mach Learn Res* 3:507–554
- Colombo D, Maathuis MH et al (2014) Order-independent constraint-based causal structure learning. *J Mach Learn Res* 15(1):3741–3782
- De Campos CP, Zeng Z, Ji Q (2009) Structure learning of bayesian networks using constraints. In: *Proceedings of the 26th annual international conference on machine learning*, pp. 113–120
- Eddelbuettel D, François R (2011) Rcpp: Seamless r and c++ integration. *J Stat Softw* 40:1–18
- Heckerman D, Geiger D, Chickering DM (1995) Learning bayesian networks: the combination of knowledge and statistical data. *Mach Learn* 20:197–243
- Jeffreys H (1998) *The theory of probability*. OUP Oxford
- Koivisto M, Sood K (2004) Exact bayesian structure discovery in bayesian networks. *J Mach Learn Res* 5(May):549–573
- Krichevsky RE, Trofimov VK (1981) The performance of universal encoding. *IEEE Trans Inf Theory* 27:199–206. <https://api.semanticscholar.org/CorpusID:2813981>
- Malone B, Yuan C, Hansen E (2011) Memory-efficient dynamic programming for learning optimal bayesian networks. *Proc AAAI Conf Artif Intell* 25:1057–1062
- Silander T, Myllymaki P (2012) A simple approach for finding the globally optimal bayesian network structure. *arXiv preprint arXiv:1206.6875*
- Singh AP, Moore AW (2005) Finding optimal Bayesian networks by dynamic programming. In: *Center for Automated Learning and Discovery*, Carnegie Mellon University
- Spirtes P, Glymour C (1991) An algorithm for fast recovery of sparse causal graphs. *Soc Sci Comput Rev* 9(1):62–72
- Suzuki J (1993) A construction of bayesian networks from databases based on an mdl principle. In: *Uncertainty in Artificial Intelligence*, Elsevier, p 266–273
- SUZUKI J (1999) Learning bayesian belief networks based on the minimum description length principle: basic properties. *IEICE Trans Fund E82-A(10)*:2237–2245
- Suzuki J, Kawahara J (2017) Branch and bound for regular bayesian network structure learning. In: *UAI*
- Yuan C, Malone B (2013) Learning optimal bayesian networks: a shortest path perspective. *J Artif Intell Res* 48:23–65
- Yuan C, Malone B, Wu X (2011) Learning optimal bayesian networks using a* search. In: *IJCAI proceedings-international joint conference on artificial intelligence*, vol 22, p 2186

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.